# Lab Assignment 11.1

| | |
|---|---|
| **Program** | **: B. Tech (CSE)** |
| **Course Title** | **: AI Assisted Coding** |
| **Course Code** | **: 23CS002PC304** |
| **Semester** | **: II** |
| **Academic Session** | **: 2025-2026** |
| **Name of Student** | **: Sruthi** |
| **Enrollment No.** | **: 2403A51L10** |
| **Batch No.** | **: 51** |
| **Date** | **: 17-02-2026** |

## Task 1:

**Sample input code:**

```
class Stack:
    pass
```

**Prompt:** Write a python program a stack class that follows the push, pop, peek and is_empty methods.

**Code:**

```
class Stack:
    def __init__(self):
        self.stack = []
    def push(self, item):
        self.stack.append(item)
    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        else:
            raise IndexError("Stack is empty")
    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        else:
            raise IndexError("Stack is empty")
    def is_empty(self):
        return len(self.stack) == 0
# Example usage:
stack = Stack()
stack.push(1)
stack.push(2)
print(stack.peek())
print(stack.pop())
print(stack.is_empty())
print(stack.pop())
print(stack.is_empty())
```

**Output:**

```
2
2
False
1
True
```

**Explanation:**

The Stack class is implemented using a list to store the elements.
The push method adds an item to the top of the stack.
The pop method removes and returns the top item.
The peek method allows you to see the top item without removing it.
The is_empty method checks if the stack is empty.

## Task 2:

**Sample code:**

```
class Queue:
    pass
```

**Prompt:** Write a python program to implement a queue.

**Code:**

```
class Queue:
    def __init__(self):
        self.queue = []
    def enqueue(self, item):
        self.queue.append(item)
    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            raise IndexError("Queue is empty")
    def peek(self):
        if not self.is_empty():
            return self.queue[0]
        else:
            raise IndexError("Queue is empty")
    def is_empty(self):
        return len(self.queue) == 0
# Example usage:
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
print(queue.peek())
print(queue.dequeue())
print(queue.is_empty())
print(queue.dequeue())
print(queue.is_empty())
```

**Output:**

```
1
1
False
2
True
```

**Explanation:**

A Queue class that uses a list to store the elements of the queue.

- The enqueue method adds an item to the end of the queue.
- The dequeue method removes and returns the item at the front of the queue.
- The peek method allows us to see the front item without removing it, and
- The is_empty method checks if the queue is empty.

## Task 3:

**Sample Code:**

```
class Node:
    pass
class LinkedList:
    pass
```

**Prompt:** Generate a Singly linked list with insert and display methods.

**Code:**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
    def display(self):
        current = self.head
        while current:
            print(current.data, end=' ')
            current = current.next
# Example usage:
linked_list = LinkedList()
linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
linked_list.display()
```

**Output:**

```
1 2 3
```

**Explanation:**
Defined a Node class to represent each node in the linked list, which contains data and a pointer to the next node. The LinkedList class contains methods to insert new nodes at the end of the list and to display the contents of the list. The insert method creates a new node and adds it to the end of the list, while the display method traverses the list and prints each node's data.

# Task 4:

**Sample Code:**

```
class BST:
    pass
```

**Prompt:** Generate a BST with insert and in-order traversal methods.

**Code:**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BST:
    def __init__(self):
        self.root = None
    def insert(self, data):
        if self.root is None:
            self.root = Node(data)
        else:
            self._insert_recursive(self.root, data)
    def _insert_recursive(self, node, data):
        if data < node.data:
            if node.left is None:
                node.left = Node(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = Node(data)
            else:
                self._insert_recursive(node.right, data)
    def inorder_traversal(self):
        return self._inorder_recursive(self.root)
    def _inorder_recursive(self, node):
        res = []
        if node:
            res = self._inorder_recursive(node.left)
            res.append(node.data)
            res = res + self._inorder_recursive(node.right)
        return res
# Example usage:
bst = BST()
bst.insert(5)
bst.insert(3)
bst.insert(7)
print(bst.inorder_traversal())
```

**Output:**

```
[3, 5, 7]
```

**Explanation:**

Defined a Node class to represent each node in the binary search tree (BST) and a BST class to manage the tree structure. The BST class has methods to insert new values and perform an in-order traversal.
The insert method adds new nodes to the tree while maintaining the BST property, and the inorder_traversal method returns a list of values in sorted order.

## Task 5:

**Sample Code:**

```
class HashTable:
    pass
```

**Prompt:** Implement a hash table with insert, search and delete methods.

**Code:**

```
class HashTable:
    def __init__(self):
        self.table = {}
    def insert(self, key, value):
        self.table[key] = value
    def search(self, key):
        return self.table.get(key, None)
    def delete(self, key):
        if key in self.table:
            del self.table[key]
# Example usage:
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))
print(hash_table.search("age"))
hash_table.delete("name")
print(hash_table.search("name"))
```

**Output:**

```
Alice
30
None
```

**Explanation:**

Define a HashTable class that uses a Python dictionary to store key-value pairs.
The insert method adds a new key-value pair to the hash table.
The search method retrieves the value associated with a given key.
The delete method removes a key-value pair from the hash table.